Python Workflows with Azure Blob Storage

Installation and Setup

1. Install the following Python dependencies via creating a `requirements.txt` file.

```
azure-storage-blob
pandas
python-dotenv
```

2. Recursively pip install.

```
pip install -r requirements.txt
```

3. Save your Azure Blob Storage connection as an environment variable inside a `.env` file.

- `.env`

```
AZURE_STORAGE_CONNECTION_STRING=...
```

Basic Usage

Getting Blob File Content into Pandas DataFrames

1. Set up the connection string and create a client.

```
import pandas as pd
from os import getcwd, environ
from io import BytesIO
from dotenv import load_dotenv
from azure.storage.blob import BlobServiceClient

# Path to the .env file
env_path = getcwd() + "/.env"

# Load environment variables
load_dotenv(env_path)

# Get the connection string from environment variables
conn_str = environ.get('AZURE_STORAGE_CONNECTION_STRING')

# Generate the service
service = BlobServiceClient.from_connection_string(conn_str)

# Name of container
container_name = "YOUR_CONTAINER_NAME"

# Access the specific container containing your CSV blobs
container = service.get_container_client(container_name)
```

2. List out the objects (blobs) within the container

```
...

# Iterate through the blobs in each container
for blob in container.list_blobs():
    # Check if the blob is a CSV file
    if blob.name.lower().endswith(".csv"):
        print(f"Found: {blob.name}")
```

3. Download the file contents into bytes

```
...

blob_name = "example.csv"

# Create a client specifically for the blob
blob_client = container.get_blob_client(blob_name)

# Get the contents as a storage stream which is Microsoft's custom buffer
stream = blob_client.download_blob()

# Get the data in bytes
data_bytes = stream.readall()
```

4. Convert the bytes object into Pandas DataFrames

```
...

# DataFrame
df = pd.read_csv(BytesIO(data_bytes))

# See the first 5 rows
print(df.head())
```

Uploading Blobs into a Container

1. Generate a `blob_client` from the `container_client`.

```
...

# Generate a name for the blob
blob_name = 'example.csv'

# Create blob client
blob_client = container.get_blob_client(blob_name)

# Read the CSV as binary

## Context Manager
csv_path = getcwd() + "/example.csv"

with open(csv_path, "rb") as data:
    # Use the method .upload_blob()
    blob_client.upload_blob(data, overwrite=True)

print(f"Blob {blob_name} uploaded to container.")
```